LÝ THUYẾT ĐỒ THỊ

knguyen – vhung – vanh

Mở đầu

Đồ thị là gì?

Đồ thị là cấu trúc toán học gồm:

$$G = (V, E)$$

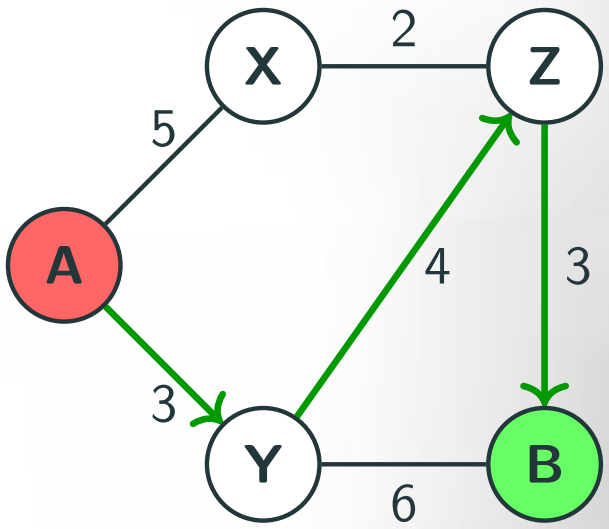
- V : tập các đỉnh
- E : tập các cạnh nối các đỉnh

Google Maps – Tìm đường đi ngắn nhất

Bài toán

Tìm đường đi ngắn nhất từ **A** đến **B**

- Các giao lộ → **Đỉnh (Vertices)**
- Các con đường → **Cạnh (Edges)**
- Thời gian / Khoảng cách → **Trọng số**

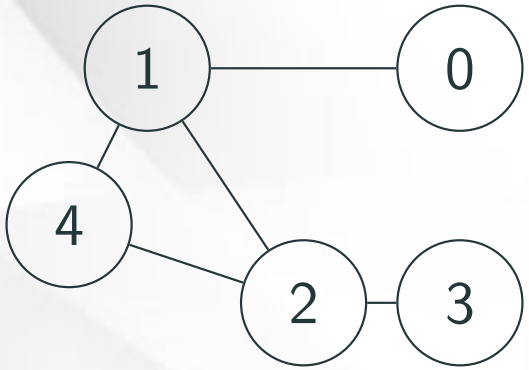


Thuật toán gợi ý của Hưng:

Sử dụng **Dijkstra** để tìm đường đi ngắn nhất

Khái niệm cơ bản

Ví dụ đồ thị



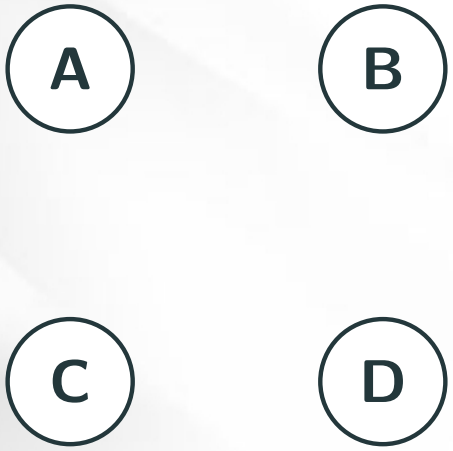
Thành phần của đồ thị

Đỉnh (Vertex)

Khái niệm

Đỉnh (vertex) là các điểm cơ bản cấu tạo nên đồ thị. Mỗi đỉnh thường biểu diễn một đối tượng.

- Ký hiệu: $V = \{A, B, C, D\}$
- Ví dụ: mỗi đỉnh là một thành phố, người, hoặc máy tính



Cạnh (Edge)

Khái niệm

Cạnh (edge) là đoạn nối giữa hai đỉnh, biểu diễn mối quan hệ giữa chúng.

- Đồ thị vô hướng: cạnh không có chiều
- Đồ thị có hướng: cạnh có mũi tên
- Đa đồ thị: có nhiều cạnh giữa 2 đỉnh

Vô hướng

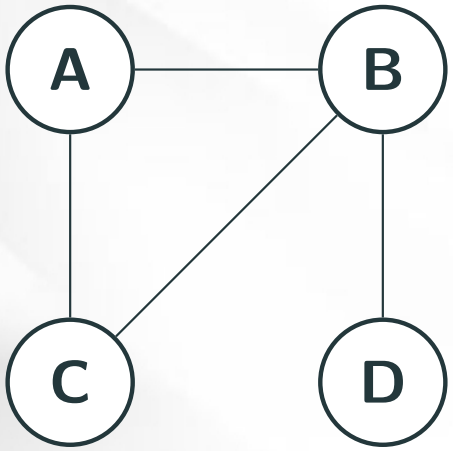
Có hướng

Bậc của đỉnh (Degree)

Khái niệm

Bậc của một đỉnh là số cạnh nối với đỉnh đó.
Với $\deg(\cdot)$ là hàm tính bậc của đỉnh, ta có:

- $\deg(A) = 2$
- $\deg(B) = 3$
- $\deg(C) = 2$
- $\deg(D) = 1$



Đường đi (Path)

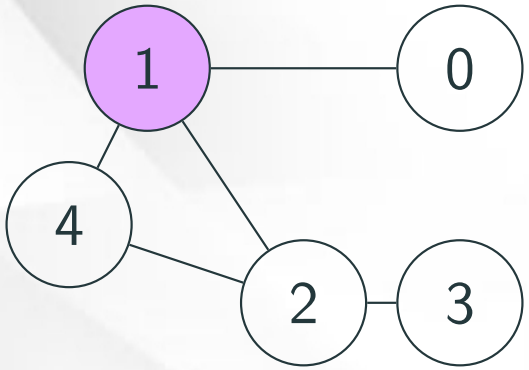
Khái niệm

Đường đi là dãy các đỉnh mà giữa các đỉnh liên tiếp đều có cạnh nối.

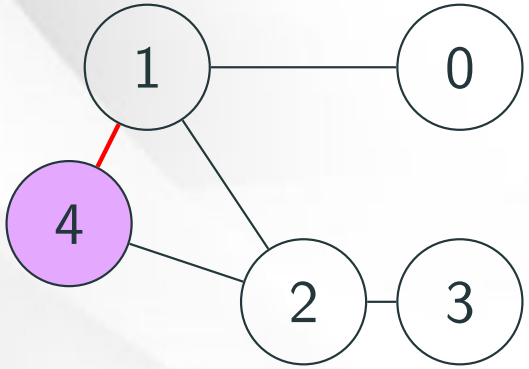
- Ví dụ: $A \to B \to C \to D$
- Độ dài đường đi = số cạnh đi qua

Đường đi

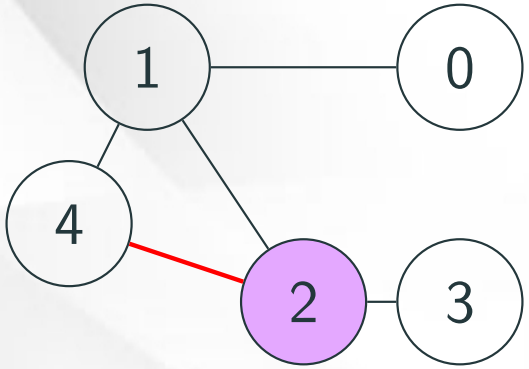
Bước 1: Bắt đầu



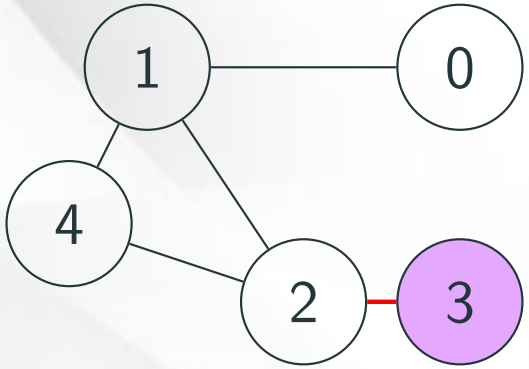
Bước 2: Di chuyển đến đỉnh 4



Bước 3: Di chuyển đến đỉnh 2



Bước 4: Di chuyển đến đỉnh 3



Cài đặt C++

Biểu diễn: Danh sách kề (Adjacency List)

Cấu trúc: Mỗi đỉnh lưu một danh sách các đỉnh láng giềng.

- **Đỉnh 1:** [4, 0, 2]
- **Đỉnh 2:** [1, 3, 4]
- **Đỉnh 3:** [2]

Ưu điểm:

- **Tiết kiệm bộ nhớ:** Chỉ tốn $O(V + E)$.
- **Duyệt nhanh:** Duyệt các đỉnh kề mất $O(deg(u))$.
- Phù hợp cho đồ thị lớn, đồ thị thưa.

Biểu diễn: Danh sách kề

Danh sách kề sử dụng mảng vector để lưu trữ các đỉnh láng giềng.

```
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;
    for(int i=0; i<m; i++){
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // Đồ thị vô hướng
    }
    return 0;
}
```

Biểu diễn: Danh sách cạnh (Edge List)

- Lưu trữ danh sách các cặp đỉnh (u, v) tạo thành cạnh.
- Rất hiệu quả khi thuật toán chỉ cần duyệt qua từng cạnh một.
  Đặc điểm:
  - Dễ cài đặt, tốn ít bộ nhớ.
  - Phù hợp với các thuật toán chỉ quan tâm đến các cạnh (như Kruskal, Bellman-Ford).
  - Không hiệu quả khi cần tìm các đỉnh kề của một đỉnh cụ thể.

Ví dụ:

STT	Đỉnh u	Đỉnh v
1	1	4
2	1	0
3	1	2
4	2	3
5	4	2

Biểu diễn: Danh sách cạnh

Danh sách cạnh lưu trữ đồ thị dưới dạng tập hợp các cặp đỉnh.

```
struct Edge {
    int u, v;
};
vector<Edge> edgeList;

// Thêm cạnh
edgeList.push_back({u, v});
```

Biểu diễn: Ma trận kề (Adjacency Matrix)

Ma trận A kích thước $N \times N$:

- $A[i][j] = 1$: Nếu có cạnh nối i và j.
- $A[i][j] = 0$: Nếu không có.

Đồ thị 3 đỉnh:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

(Dòng 1 nối với đỉnh 2, Dòng 2 nối 1 và 3, v.v.)

Nhược điểm: Tốn bộ nhớ $O(N^2)$, không tối ưu với đồ thị có hàng chục ngàn đỉnh.

Biểu diễn: Ma trận kề

```
// Ma trận kích thước tối đa 100x100
int adj[100][100];
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        // Đồ thị vô hướng: đánh dấu 1 ở cả 2 chiều
        adj[u][v] = 1;
        adj[v][u] = 1;
    }
    return 0;
}
```

Thuật toán DFS

Thuật toán DFS (Depth-First Search)

- **Nguyên lý:** "Đi sâu nhất có thể"trước khi quay lui (Backtracking).
- **Cấu trúc dữ liệu:** Sử dụng **Ngăn xếp (Stack)** hoặc **Đệ quy** (chính là Stack của hệ thống).
- **Các bước:**
  1. Thăm đỉnh u, đánh dấu u là đã thăm (visited).
  2. Với mỗi đỉnh v kề với u:
     - Nếu v chưa được thăm, thực hiện gọi đệ quy `DFS(v)`.

Lưu ý: DFS rất dễ cài đặt nhờ đệ quy, nhưng cần cẩn thận với đồ thị quá lớn vì có thể gây tràn Stack (Stack Overflow).

Cài đặt DFS với C++ (Đệ quy)

```
vector<int> adj[100]; // Danh sách kề
bool visited[100];    // Mảng đánh dấu đã thăm

void DFS(int u) {
    visited[u] = true;       // Đánh dấu u đã thăm
    // Duyệt qua các đỉnh kề v
    for(int v : adj[u]) {
        if(!visited[v]) {
            DFS(v);          // Gọi đệ quy cho v
        }
    }
}
```

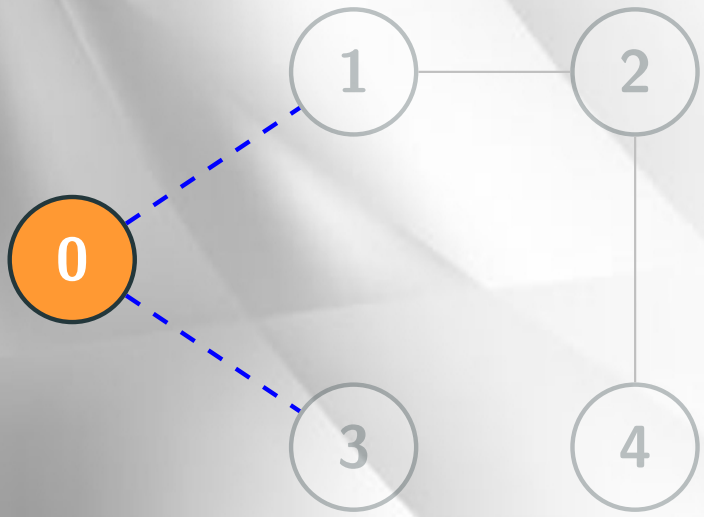
Ghi chú: visited[] đóng vai trò cực kỳ quan trọng để ngăn chương trình lặp vô tận (nếu đồ thị có chu trình).

Thuật toán BFS

4. BFS: Tìm kiếm theo chiều rộng

- **Ý Tưởng:** Duyệt theo từng "lớp"vết dầu loang. Thăm hết tất cả các đỉnh kề trực tiếp rồi mới tiến ra xa hơn.
- **Đặc điểm:** Luôn tìm được đường đi ngắn nhất trong đồ thị không trọng số.
- **Độ phức tạp:** Thời gian $O(V + E)$.

Mô phỏng từng bước BFS

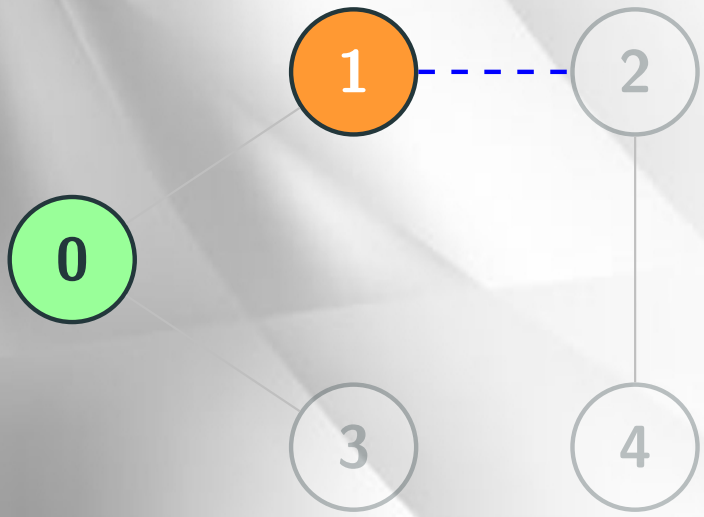


Hàng đợi Queue:

1	3

B1:
-Thăm 0
-Đưa đỉnh kề {1, 3} vào Queue.

Mô phỏng từng bước BFS

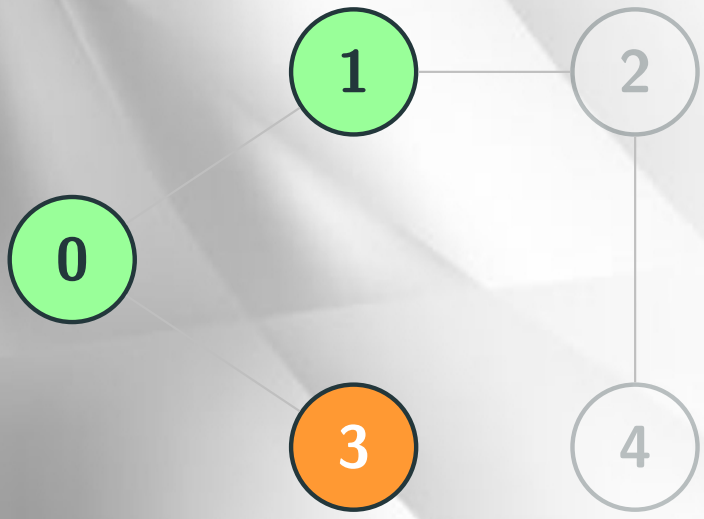


Hàng đợi Queue:

3	2

B2:
-Lấy 1 ra thăm.
-Thêm đỉnh kề 2 vào Queue.

Mô phỏng từng bước BFS

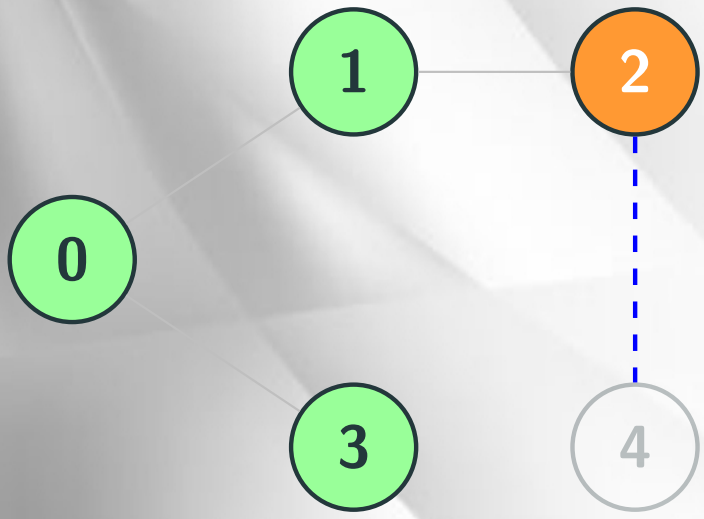


Hàng đợi Queue:

2

B3:
-Lấy 3 ra thăm.
-Đỉnh 0 đã được duyệt.

Mô phỏng từng bước BFS

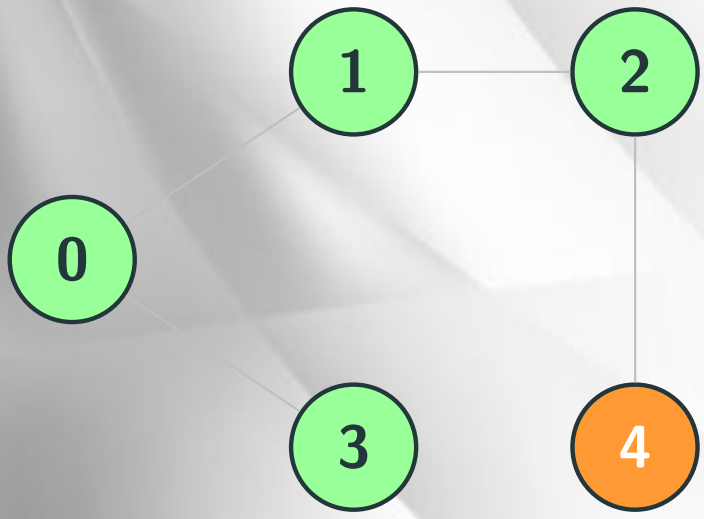


Hàng đợi Queue:

4

B4:
-Lấy 2 ra thăm.
-Thêm đỉnh kề 4 vào Queue.

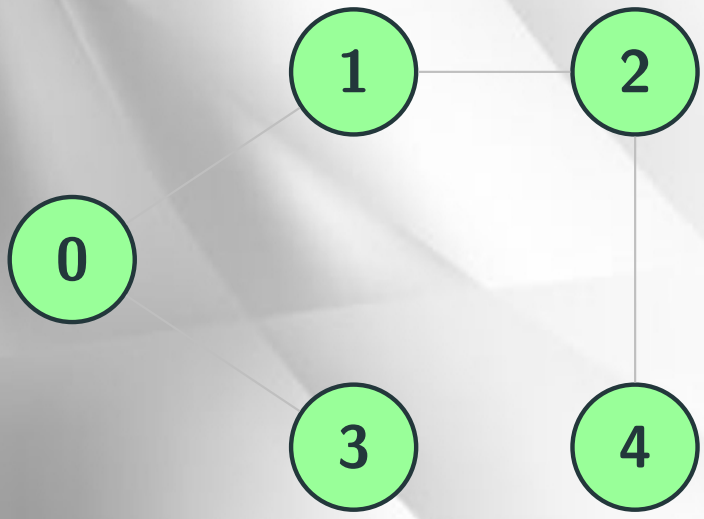
Mô phỏng từng bước BFS



Hàng đợi Queue:

B5:
-Lấy 4 ra thăm.
-Hoàn thành.

Mô phỏng từng bước BFS



Hàng đợi Queue:

Kết quả:
$0 \to 1 \to 3 \to 2 \to 4.$

Mã nguồn BFS (C++)

```
void bfs(int start_node) {
    queue<int> q;
    q.push(start_node);
    visited[start_node] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

CÂY

Cây (Tree)

Định nghĩa

- Cây là một đồ thị:
  - Liên thông
  - Không có chu trình
- Nếu có n đỉnh thì có đúng $n - 1$ cạnh

Đặc điểm quan trọng:

- Giữa 2 đỉnh bất kỳ chỉ có **1 đường đi duy nhất**

Thuật ngữ trong cây

- **Root (gốc):** đỉnh bắt đầu (thường là 1)
- **Parent (cha):** đỉnh phía trên
- **Child (con):** đỉnh phía dưới
- **Leaf (lá):** đỉnh có bậc $= 1$
- **Depth:** khoảng cách từ gốc
- **Height:** độ sâu lớn nhất của cây

Biểu diễn cây

Danh sách kề (Adjacency List)

- Giống đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề

Ví dụ:

- $1 : 2, 3$
- $2 : 1, 4$
- $3 : 1$

Duyệt cây

Hai cách chính:

- **DFS (Depth First Search)**
  - Đi sâu trước
- **BFS (Breadth First Search)**
  - Đi theo từng lớp

Ứng dụng:

- Tính khoảng cách
- Tìm đường đi
- Xử lý cây

Tính chất của cây

- n đỉnh $\Rightarrow$ $n-1$ cạnh
- Không có chu trình
- Luôn liên thông
- Có ít nhất 2 đỉnh lá (n $>$ 1)

Hệ quả:

- Có thể dùng BFS/DFS để duyệt toàn bộ cây

Các bài toán thường gặp

- Tính độ sâu của các đỉnh
- Tìm đường kính của cây
- Đếm số lá
- Tìm cha, tổ tiên
- Khoảng cách giữa 2 đỉnh

Mức nâng cao:

- LCA (Lowest Common Ancestor)
- Tree DP

Ứng dụng

Ứng dụng của đồ thị

- Google Maps (Tìm đường đi ngắn nhất)
- Mạng xã hội (Mối quan hệ)
- Internet (Liên kết trang web)
- Game AI (Di chuyển nhân vật)

Bài tập

PHẦN BÀI TẬP

Luyện tập áp dụng BFS

Bài 1

Xe tải của Việt Hưng

Một thành phố có n ngôi nhà và m đoạn đường hai chiều nối giữa các ngôi nhà, với mỗi đoạn đường, người ta tính được khối lượng tối đa để xe có thể đi qua mà không gây hư hại cho đoạn đường đó.

Xe tải của Hưng cần chở x kg hàng hoá từ ngôi nhà s đến ngôi nhà t. Kiểm tra xem Hưng có thể thực hiện công việc được không.

Input

- Dòng đầu tiên chứa năm số nguyên n, m, x, s, t
- m dòng tiếp theo, mỗi dòng là ba số nguyên x, y, w

Bài 1

Output

- Nếu đi được từ s đến t, in ra YES, ngược lại in ra NO

Giới hạn

- $1 \leq n, m \leq 10^5$
- $1 \leq s, t \leq n$
- $1 \leq x, y \leq n$
- $1 \leq w \leq 10^9$

Bài 1

Ý tưởng

Lọc các cạnh có $w \geq x$ và dùng BFS kiểm tra liên thông.

Code

```
#include<bits/stdc++.h>
using namespace std;

bool is_visited[1000006];
vector<pair<long long,long long>> adj[1000006];

void dfs(long long u,long long x){
    is_visited[u]=true;
    for(auto [v,l]:adj[u]){
        if(!is_visited[v] && l>=x){
            dfs(v,x);
        }
    }
}

bool check(long long mid,long long s,long long t){
    memset(is_visited,false,sizeof(is_visited));
    dfs(s,mid);
    return is_visited[t];
}
```

Bài 1

```
long long tknp(long long s,long long t){
    long long l=1,r=1e9,kq=-1;
    while(l<=r){
        long long mid=(l+r)/2;
        if(check(mid,s,t)){
            kq=mid;
            l=mid+1;
        } else {
            r=mid-1;
        }
    }
    return kq;
}
```

Bài 1

```
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    long long n,m,s,t;
    cin>>n>>m>>s>>t;
    for(long long i=0;i<m;i++){
        long long x,y,l;
        cin>>x>>y>>l;
        adj[x].push_back({y,l});
        adj[y].push_back({x,l});
    }

    cout<<tknp(s,t);
}
```

Bài 2

Bàn cờ

Xét bàn cờ vua có kích thước $n \times n$. Các dòng được đánh số từ 1 đến n, từ trên xuống dưới. Các cột được đánh số từ 1 đến n, từ trái qua phải. Ô nằm trên giao của dòng i và cột j được gọi là ô (i, j).
Trên bàn cờ có m $(0 \leq m \leq n^2 - 2)$ quân cờ. Với $m > 0$, quân cờ thứ i ở ô (r_i, c_i), $i = 1, \ldots, m$. Không có hai quân cờ nào ở cùng vị trí.

Bài 2

Trong số các ô còn lại của bàn cờ, tại ô (p, q) có 1 quân xe. Mỗi một nước đi, từ vị trí đang đứng thì quân xe chỉ có thể di chuyển đến được những ô trên cùng hàng hoặc cùng cột mà trên đường đi không phải đi qua các ô đã có quân.
Cần phải đưa quân xe từ ô xuất phát (p, q) về ô đích (s, t). Giả thiết là ô đích không có quân cờ.

Input

- Dòng đầu tiên chứa 6 số nguyên n, m, p, q, s, t
- m dòng tiếp theo là vị trí các quân cờ

Bài 2

Output

- Một số nguyên là số nước đi ít nhất. Nếu không đi được, in ra -1

Giới hạn

- $1 \leq n \leq 200$
- $0 \leq m \leq n^2 - 2$

Bài 2

Ý tưởng

- Xem mỗi ô là một đỉnh
- BFS trên ma trận
- Từ mỗi ô, đi theo 4 hướng như quân xe
- Dừng khi gặp biên hoặc quân cản

Bản chất: BFS trên lưới với bước đi đặc biệt

Bài 2 - Code (1/2)

```
#include<bits/stdc++.h>
using namespace std;

long long dps[1003][1003], mtk[1003][1003];

void bfs(int x1,int y1,int s,int t,int n){
    memset(dps,-1,sizeof(dps));
    queue<pair<int,int>> q;

    q.push({x1,y1});
    dps[x1][y1]=0;

    while(!q.empty()){
        int u=q.front().first;
        int v=q.front().second;
        q.pop();
```

Bài 2 - Code (2/2)

```
        const int dx[]={0,0,1,-1};
        const int dy[]={1,-1,0,0};

        for(int i=0;i<4;i++){
            int x=u,y=v;

            while(true){
                x+=dx[i];
                y+=dy[i];

                if(x<1 || x>n || y<1 || y>n || mtk[x][y]==1) break;

                if(dps[x][y]==-1){
                    dps[x][y]=dps[u][v]+1;
                    q.push({x,y});
                }
            }
        }
    }

    cout<<dps[s][t];
}
```

Bài 2 - Code (3/3)

```
int main(){
    ios::sync_with_stdio(0);
    cin.tie(0);

    long long n,m,p,q,s,t;
    cin>>n>>m>>p>>q>>s>>t;

    for(int i=1;i<=m;i++){
        long long x,y;
        cin>>x>>y;
        mtk[x][y]=1;
    }

    bfs(p,q,s,t,n);
}
```

Bài 3

Đường kính của cây

Cho một cây gồm n đỉnh và $n-1$ cạnh.
Hãy tìm độ dài đường đi dài nhất giữa hai đỉnh bất kỳ trong cây.

Input

- Số nguyên n
- $n-1$ dòng mỗi dòng là cạnh u, v

Bài 3

Output

- Độ dài đường kính của cây

Giới hạn

- $1 \leq n \leq 2 \times 10^5$

Bài 3

Ý tưởng

- BFS từ một đỉnh bất kỳ $\rightarrow$ tìm đỉnh xa nhất u
- BFS từ $u \rightarrow$ tìm đỉnh xa nhất v

Khoảng cách $u \rightarrow v$ chính là đường kính

Bài 3 - Code

```
vector<int> adj[200005];
int dist[200005];

int bfs(int s){
    memset(dist,-1,sizeof(dist));
    queue<int> q;
    q.push(s);
    dist[s]=0;
    int last = s;
    while(!q.empty()){
        int u=q.front(); q.pop();
        last = u;

        for(int v:adj[u]){
            if(dist[v]==-1){
                dist[v]=dist[u]+1;
                q.push(v);
            }
        }
    }

    return last;
}
```

Bài 4

Đếm số lá

Cho một cây gồm n đỉnh.
Hãy đếm số đỉnh lá của cây (đỉnh có đúng 1 cạnh, trừ trường hợp $n = 1$).

Input

- Số nguyên n
- $n - 1$ cạnh

Bài 4

Output

- Số lượng đỉnh lá

Giới hạn

- $1 \leq n \leq 2 \times 10^5$

Bài 4

Ý tưởng

- Đỉnh lá là đỉnh có bậc $= 1$
- Duyệt tất cả các đỉnh
- Đếm số đỉnh có degree $= 1$

Lưu ý: nếu $n = 1$ thì kết quả $= 1$

Bài 4 - Code

```
int main(){
    int n;
    cin>>n;

    vector<int> deg(n+1,0);

    for(int i=1;i<n;i++){
        int u,v;
        cin>>u>>v;
        deg[u]++;
        deg[v]++;
    }
    if(n==1){
        cout<<1;
        return 0;
    }
    int cnt=0;
    for(int i=1;i<=n;i++){
        if(deg[i]==1) cnt++;
    }

    cout<<cnt;
}
```

LỜI CHÀO

CẢM ƠN